COS 350 Lecture 10

unix: find

unix: which

bash: type

case study: ls

- opendir
- readdir
- closedir
- scandir

FIND(1) FIND(1)

NAME

find - search for files in a directory hierarchy

SYNOPSIS

find [path...] [expressions and actions]

DESCRIPTION

Find searches the directory tree rooted at each given starting-point by evaluating the given expression.

EXPRESSIONS

-name <u>pattern</u>	Search for file matching the shell pattern.
-size [+-]<u>n</u>[unit]	Search for file by size - units: bcwkMG
-perm <u>mode</u>	Search by access permisions

ACTIONS

-print	Display names of matching files (default action)
-exec <u>CMD</u> {} \;	Execute CMD on matching files.
	{} is replaced by each matching file name
	\; ends the command

Example 1: find files with exact name

```
unix> find ./submit -name
myexpand.c
./submit/cchipman/prog1/myexpand.
c
./submit/rgillen/prog1/myexpand.c
./submit/christy/prog1/myexpand.c
./submit/mjewell/prog1/myexpand.c
./submit/ferland/prog1/myexpand.c
./submit/mschell/prog1/myexpand.c
./submit/jaltman/prog1/myexpand.c
./submit/alamarre/prog1/myexpand.
c
./submit/washburn/prog1/myexpand.
c
./submit/kcronin/prog1/myexpand.c
./submit/dhall/prog1/myexpand.c
./submit/dcarlson/prog1/myexpand.
c
./submit/knowlton/prog1/myexpand.
c
./submit/afoster/prog1/myexpand.c
```

Example 2: find files with wildcard name

```
unix> find ./submit -name '*.c'
./submit/rgillen/prog1/myexpand.c
./submit/boothe/testingprog3/qsort.c
./submit/nrosen/prog1/myExpand.c
./submit/christy/prog1/myexpand.c
./submit/bartolom/prog1/main.c
./submit/mjewell/prog1/myexpand.c
./submit/ferland/prog1/myexpand.c
./submit/mschell/prog1/myexpand.c
./submit/mschell/prog2/z827.c
./submit/jaltman/prog1/myexpand.c
./submit/alamarre/prog1IGNOREOTHER/myexpand.c
./submit/alamarre/prog1/myexpand.c
./submit/washburn/prog1/myexpander.c
./submit/washburn/prog1/myexpand.c
./submit/kcronin/prog1/myexpand.c
./submit/dhall/prog1/myexpand.c
./submit/dcarlson/prog1/myexpand.c
…
```

Example 3: find files and execute a command on them

```
unix> find ./submit -name myexpand.c -exec wc {} \;
 lines  words  chars
   47    116    766 ./submit/aaaaaaa/prog1/myexpand.c
   95    264   2328 ./submit/bbbbb/prog1/myexpand.c
   44    183   1101 ./submit/ccccccc/prog1/myexpand.c
  106    247   1732 ./submit/dddddd/prog1/myexpand.c
   63    124    880 ./submit/eeeeee/Prog1/myexpand.c
   81    249   1567 ./submit/fffffff/prog1/myexpand.c
  100    331   2094 ./submit/ggggggg/prog1/myexpand.c
   62    339   2189 ./submit/hhhhhhh/program#1/myexpand.c
  150    668   4185 ./submit/iiiii/prog1/myexpand.c
   36    100    718 ./submit/jjjjjj/prog1/myexpand.c
   66    198   1303 ./submit/kkkkkkk/prog1/myexpand.c
   63    252   1817 ./submit/lllllll/prog1/myexpand.c
   38    181   1081 ./submit/mmmmm/prog1/myexpand.c
   60    154   1341 ./submit/nnnnnnn/prog1/myexpand.c
   90    194   1286 ./submit/ooooooo/prog1/myexpand.c
...
```

Example 4: find files and search them with grep

```
unix> find . -name '*.c' -exec grep sbrk {} \; -print
```

search files *print file name*

```
// mymalloc0.c - demonstrates sbrk()
  return sbrk(size);
./public_www/cos350/courseExamples/l26/mymalloc0.c
      if ((chunk = sbrk(chunksize)) == (void*)-1)
//        brk((void*)(((long)sbrk(0)+7) & ~7)); // set brk to align
./public_www/cos350/courseExamples/l26/mymalloc1.c
      if ((chunk = sbrk(chunksize)) == (void*)-1)
//        brk((void*)(((long)sbrk(0)+7) & ~7)); // brk to x8
./public_www/cos350/courseExamples/l26/mymalloc2.c
...
```

WHICH(1) WHICH(1)

NAME

which - locate a command

SYNOPSIS

which [-a] filename ...

DESCRIPTION

which returns the pathnames of the files (or links) which would be executed in the current environment, had its arguments been given as commands in a strictly POSIX-conformant shell. It does this by Searching the PATH for executable files matching the names of the arguments. It does not follow symbolic links.

OPTIONS

-a print all matching pathnames of each argument

```
unix> which split
/usr/bin/split
```

BASH(1) General Commands Manual

…

SHELL BUILTIN COMMANDS

…

type name

Indicates how name would be interpreted if used as a command name.

…

```
bash> type split
split is /usr/bin/split

bash> type myls
myls is ./myls

bash> type ls
ls is aliased to `ls -F'

bash> type type
type is a shell builtin
```

Case study: ls

```
unix> ls
ls1  ls1b  ls1b.c  ls1.c  ls1.c~  tempdir

unix> ls -l
total 40
-rwx------ 1 boothe boothe 8848 Feb 24 12:47 ls1
-rwx------ 1 boothe boothe 8668 Feb 26  2014 ls1b
-rw------- 1 boothe boothe  955 Feb 26  2014 ls1b.c
-rw------- 1 boothe boothe  774 Feb 24 12:45 ls1.c
-rw------- 1 boothe boothe  774 Feb 25  2010 ls1.c~
drwx------ 2 boothe boothe 4096 Feb 25  2010 tempdir
```

NAME

opendir - open a directory

SYNOPSIS

```
#include <sys/types.h>
#include <dirent.h>

DIR *opendir(const char *name);
```

DESCRIPTION

The opendir() function opens a directory stream corresponding to the directory name, and returns a pointer to the directory stream. The stream is positioned at the first entry in the directory.

RETURN VALUE

The opendir() function return a pointer to the directory stream.
On error, NULL is returned, and errno is set appropriately.

ERRORS

EACCES Permission denied.
ENOENT Directory does not exist, or name is an empty string.

NAME

readdir - read a directory

SYNOPSIS

```
#include <dirent.h>
struct dirent *readdir(DIR *dirp);
```

DESCRIPTION

The readdir() function returns a pointer to a dirent structure representing the next directory entry in the directory stream pointed to by dirp. It returns NULL on reaching the end of the directory stream or if an error occurred.

On Linux, the dirent structure is defined as follows:

```
struct dirent {
    ino_t          d_ino;       /* inode number */
    off_t          d_off;       /* offset to the next dirent */
    unsigned short d_reclen;    /* length of this record */
    unsigned char  d_type;      /* type of file (linux & bsd unix) */
    char           d_name[256]; /* filename */
};
```

NAME

closedir - close a directory

SYNOPSIS

```
#include <sys/types.h>

#include <dirent.h>

int closedir(DIR *dirp);
```

DESCRIPTION

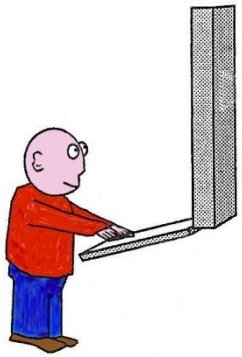
The closedir() function closes the directory stream associated with dirp. A successful call to closedir() also closes the underlying file descriptor associated with dirp. The directory stream descriptor dirp is not available after this call.

RETURN VALUE

The closedir() function returns 0 on success. On error, -1 is returned, and errno is set appropriately.

ERRORS

EBADF Invalid directory stream descriptor dirp.



ls1.c

scandir - scan a directory for matching entries

SYNOPSIS

```
#include <dirent.h>

int scandir(const char *dirname, struct dirent ***namelist,
      int (*filter)(const struct dirent *),
      int (*compar)(const struct dirent **, const struct dirent **));

int alphasort(const void *a, const void *b);
```

DESCRIPTION

The scandir() function scans the directory dirname, calling filter() on each directory entry. Entries for which filter() returns non-zero are stored in structs allocated via malloc(3), sorted using qsort(3) with the comparison function compar(), and collected in array namelist which is allocated via malloc(3). If filter is NULL, all entries are selected.

The alphasort() function can be used as the comparison function compar().

RETURN VALUE

Returns the number of directory entries selected or -1 if an error occurs.

example: using scandir

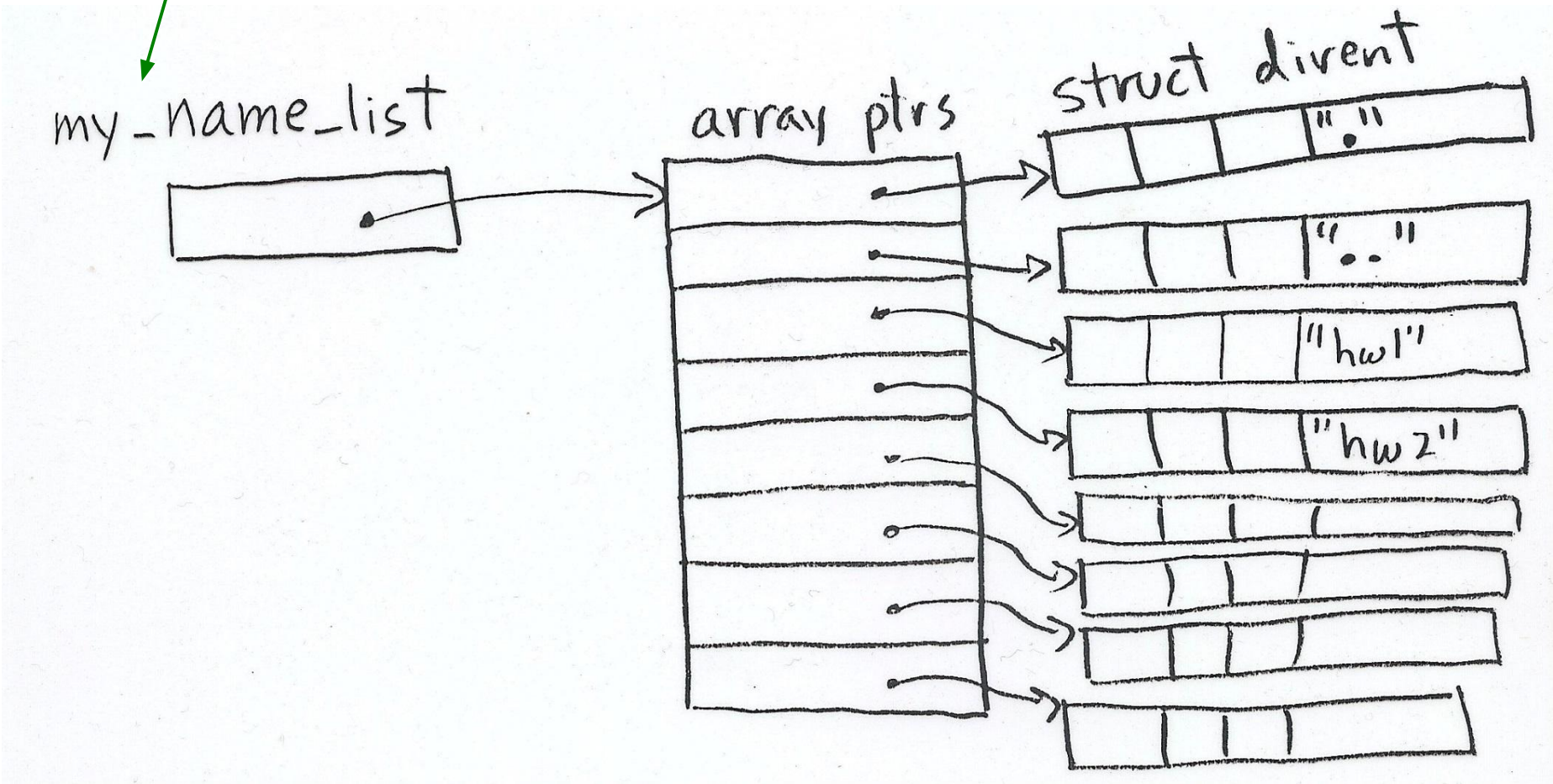
```
int n;
struct dirent **my_name_list;

n = scandir(".", &my_name_list, NULL, alphasort);
```

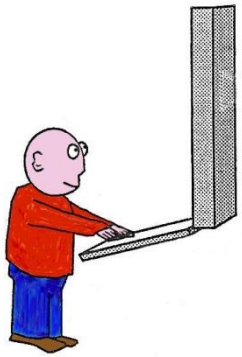
select all files

sort alphabetically

returns number of entries

scandir creates array of struct pointers





ls1b.c - using scandir & sorting by name

add filter for hidden files